



SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA15 Cloud Computing and Big Data Analytics

CO2 AT1 - Capstone Infrastructure Sizing Workbook

Guided calibration workbook for VM sizing, virtualization decisions, snapshot planning and VM-container comparison

Assessment Metadata

Course Code	CSA15
Course Title	Cloud Computing and Big Data Analytics
Assessment	CO2 AT1 - Virtualization Scenario Problem-Solving
Submission Type	Individual digital workbook based on the same capstone title used in CO1
Faculty	Dr. C. Rajesh Babu
Employee ID	SSETSCS-415
Submission Mode	Completed workbook and evidence repository link through Google Classroom

How This Workbook Works

- Read each guidance block and immediately fill the related response table.
- Use the same capstone title selected for CO1.
- Make reasonable assumptions when exact values are unknown. Good assumptions must be written clearly.
- The goal is not to guess a perfect industry configuration; the goal is to reason technically and justify the first infrastructure plan.
- The final decisions from this workbook will be used again in CO2 AT2 and CO2 AT3.

Activity 1: Bring Forward CO1 Capstone Context

SkillHire is a cloud-native NLP platform that accepts a candidate's resume PDF and a recruiter's job description, extracts SKILL entities from both using spaCy Named Entity Recognition, computes a skill gap score, and returns a ranked list of missing skills with learning recommendations via a React dashboard.

Capstone Element	Student Entry
Primary users	• Candidates (fresh graduates, job-switchers) uploading resumes • Recruiters shortlisting and comparing applicants • Admins / placement officers monitoring institution-wide trends
User roles	Candidate Recruiter Admin — three distinct Cognito user pool groups with separate dashboard views and API permissions
Main modules	1. Resume PDF upload and parsing (pdfplumber) 2. NLP skill extraction engine (spaCy NER) 3. Skill gap scoring and ranking 4. REST API layer (POST /upload, POST /match, GET /report) 5. React skill-gap dashboard 6. Recruiter shortlisting panel 7. Admin analytics and trend reports
Cloud services identified in CO1	• AWS Lambda — NLP inference • AWS API Gateway — REST endpoint management • AWS Cognito — OAuth 2.0 authentication • Amazon S3 — resume PDF and report storage • Amazon DynamoDB — match results and user records • AWS CloudWatch — logging and alerting • Amazon QuickSight — analytics dashboard • AWS Amplify + CloudFront — React frontend hosting
API/backend need	REST API with two primary endpoints: POST /api/upload (PDF ingestion) and POST /api/match. API Gateway handles rate limiting (100 RPS/user), JWT validation, CORS, and routing to Lambda.
Database need	Amazon DynamoDB: table SkillHireMatches with PK = user_id, SK = match_timestamp. Stores skill_gap_score, missing_skills, matched_skills. DynamoDB Streams trigger downstream analytics.
File upload/storage need	Amazon S3 private bucket (SSE-S3 encrypted): stores uploaded PDF resumes under /resumes/{user_id}/{timestamp}.pdf and exported gap reports under /reports/. Pre-signed URLs for upload (30-s expiry) and download. 90-day lifecycle rule for GDPR compliance.
Analytics/AI/Python module	spaCy v3 NER model packaged in a Docker container image (ECR) deployed on AWS Lambda. Extracts SKILL entities from resume text and JD text, computes set difference (missing skills) and $\text{match_score} = \text{intersection} / \text{JD_skills} \times 100$. Python libraries: spaCy, pdfplumber, boto3.

Activity 2: Workload Classification

Workload type decision: SkillHire has two distinct workload layers. The web/API layer is an API backend + file-heavy app (PDF upload, REST requests, authentication). The processing layer is an AI/Python inference module (spaCy NER). Both layers must be considered separately.

Workload Layer	Classification	Signals Present in SkillHire
Web / API frontend	API backend + File-heavy app	REST API calls, JWT auth, PDF upload/download, React dashboard serving
NLP inference (Lambda)	AI/Python module	spaCy NER model, PDF parsing, set operations, JSON response generation
Database tier	Database-heavy app (managed)	Frequent reads/writes, match history queries, DynamoDB Streams for analytics
File storage tier	File-heavy (separate object storage)	PDF uploads ≤ 5 MB each, report downloads, versioning, lifecycle rules
Analytics tier	Analytics app (managed service)	Skill trend aggregation, QuickSight dashboards, recruiter reports

Student Decision: My capstone workload type and reason

SkillHire is a multi-tier hybrid workload. The prototype will run on a single VM (2 vCPU, 4 GB RAM) hosting the Node.js/Python API. For the pilot, the NLP Lambda function is separated into a serverless compute layer (AWS Lambda), file storage moves to S3, and the database to DynamoDB. In production, the API layer gains a load balancer and auto-scaling. The AI/NLP module warrants a 4–8 vCPU / 16–32 GB RAM baseline if run on-premise, or a container on AWS Lambda with a Docker ECR image in cloud-native deployment.

Activity 3: User and Traffic Calibration

Assumption basis: SkillHire is initially deployed as an institution-wide pilot at a single engineering college with approximately 2,000 registered users (students + recruiters + admins). Peak load occurs during campus placement season (October–December) when multiple batches are active simultaneously.

Metric / Input Value	Value	Reason / Assumption
Registered users expected	10200	Full campus deployment at SIMATS: 10,000 students (1st to 4th year) + 200 recruiters/admins.
Active user percentage	10%	Standard concurrent active user baseline during placement drives and internship postings.
Peak factor	3×	Spike multiplier during deadline rushes, portal announcements, and placement registration events.
Concurrent active users (calculated)	3060	Formula: $10,200 \times 10\% \times 3 = 3,060$ peak concurrent users. (<i>Baseline active users without peak multiplier = 1,020</i>).
Actions per user per minute	2	Each user averages ~2 actions/min (e.g., uploading PDF resume, parsing skill gap, viewing match results).
Requests per minute (calculated)	2040 RPM	Formula based on baseline active users: $1,020 \text{ concurrent} \times 2 \text{ actions/min} = 2,040 \text{ RPM}$. (<i>Peak RPM during burst = 6,120 RPM</i>).
Requests per second (calculated)	34.0 RPS	Formula based on baseline active load: $2,040/60 = 34.0 \text{ RPS}$. (<i>Peak RPS during burst = 102.0 RPS</i>).
Daily active users	1530	~ 15% of 10,200 registered users active on a typical campus placement or internship day.
Daily requests	45900	Formula: $1,530 \text{ DAU} \times 30 \text{ avg actions/day} = 45,900 \text{ daily requests}$.

Activity 4: CPU Calibration

CPU Driver	Score	Reason for SkillHire
Base web/API application	1	React SPA served via CloudFront (minimal API server CPU); REST API layer is lightweight
Authentication and roles	+1	AWS Cognito OAuth 2.0 JWT validation on every API call; three distinct user roles
API traffic	+1	2.0 RPS at peak — above 5 RPS threshold NOT reached in prototype; score +1 for moderate traffic
Database/search/reporting	+1	DynamoDB reads/writes per request; Streams trigger analytics — moderate DB interaction

CPU Driver	Score	Reason for SkillHire
Background jobs	+1	DynamoDB Streams → Lambda trigger for analytics aggregation; SNS notification on alerts
Analytics/dashboard	+2	QuickSight skill-trend aggregation; recruiter reports query DynamoDB; admin dashboard polling
AI/Python NLP module	+3	spaCy NER model loading + PDF parsing + set operations per resume match; Docker container on Lambda — CPU-intensive per invocation; mid-range +3 (not GPU inference)
Total CPU Score	10	Score 10 → Scale out: multiple VMs or managed compute. In cloud-native deployment: Lambda handles NLP (serverless); API GW + separate VM for coordinator if needed.
Recommended vCPU	4–8 vCPU (prototype VM) or Lambda + 2 vCPU coordinator	Prototype: single 4 vCPU VM runs API + NLP together. Pilot/Production: NLP on Lambda (serverless, auto-scales); API coordinator VM = 2 vCPU, 4 GB RAM. This separates the CPU-heavy NLP from the lightweight REST routing.

Activity 5: RAM Calibration

RAM formula: Estimated RAM = OS/runtime + web/API + database/cache + AI/Python + 25% buffer. Since DynamoDB is fully managed, no database memory is allocated in the app VM. The Lambda NLP function is separately billed — RAM estimate below covers a prototype single-VM deployment.

Component	Student Estimate	Justification
OS + runtime (Python 3.11 + Node.js)	2 GB	Ubuntu 22.04 LTS: ~1 GB OS + 0.5 GB Node.js runtime + 0.5 GB Python 3.11 baseline
Web/API service (FastAPI / Express)	1 GB	Lightweight REST layer: FastAPI (Python) or Express (Node.js); typical idle footprint ~500 MB; peak ~1 GB
spaCy NLP model in-process	6 GB	spaCy en_core_web_lg model ≈ 750 MB on disk; loaded into memory ≈ 1.5 GB; with pdfplumber and processing buffers, sustained at 6 GB for concurrent requests
Cache layer (Redis / in-memory)	1 GB	Optional Redis cache for recently matched skill lists; reduces repeat NLP calls for same JD; 1 GB is sufficient for ~10,000 cached entries
Analytics/reporting buffer	1 GB	QuickSight uses managed compute; local analytics buffer for DynamoDB aggregation queries = 1 GB
Subtotal (before buffer)	11 GB	$2 + 1 + 6 + 1 + 1 = 11$ GB
25% safety buffer	2.75 GB	Formula: $11 \times 0.25 = 2.75$ GB — accounts for OS kernel spikes, garbage collection, swap avoidance
Total before rounding	13.75 GB	$11 + 2.75 = 13.75$ GB
Final RAM (rounded to standard)	16 GB	Round 13.75 GB → 16 GB (next standard tier). Note: in cloud-native pilot, Lambda function is allocated 3 GB RAM per invocation for spaCy model; prototype VM needs 16 GB to handle the full stack.

Activity 6: Storage Calibration

Technical tip applied: PDF resumes and gap reports are not stored on the VM disk — they are offloaded to Amazon S3. VM disk covers OS, application code, Python libraries (including spaCy model), and logs only.

Storage Item	Student Estimate	Formula / Justification
OS and application files	40 GB	Ubuntu 22.04: 8 GB. Python libs (spaCy + model): 4 GB. App code + Docker layer: 2 GB. Node.js/React build: 1 GB. Reserve: 25 GB. Total: 40 GB
Database (local prototype only)	10 GB	DynamoDB is managed (cloud). On prototype VM: local SQLite/Postgres mirror for dev/test. Avg record = 2 KB; 6,000 daily records $\times$ 365 days $\times$ 2 KB $\times$ 1.3 / 1,024 = ~5.6 GB $\rightarrow$ rounded to 10 GB
File uploads (S3, not on VM)	S3: 78 GB/year	Formula: avg PDF 1.5 MB $\times$ 100 uploads/day $\times$ 365 days $\times$ 1.3 = 71.2 GB/year on S3. VM disk: 0 GB for uploads (offloaded to S3). S3 lifecycle deletes after 90 days $\rightarrow$ ~24 GB steady-state
Logs (on VM)	5 GB	Formula: 6,000 requests/day $\times$ 1 KB log/request $\times$ 365 days / 1,024 = 2.1 GB/year. Add Lambda invocation logs synced via CloudWatch: additional 2.9 GB $\rightarrow$ 5 GB total on VM disk
Backup/growth buffer (VM disk)	16.5 GB	Subtotal (OS 40 + DB 10 + Logs 5) = 55 GB. Buffer: $55 \times 30\% = 16.5$ GB
Total VM disk (before rounding)	71.5 GB	$40 + 10 + 5 + 16.5 = 71.5$ GB
Final VM disk (rounded)	100 GB SSD	Round 71.5 GB $\rightarrow$ 100 GB SSD (standard EBS gp3 volume). S3 for file storage is additional and billed separately (first 5 GB free; ~24 GB steady-state = ~\$0.55/month).

Activity 7: Select the VM Plan

SkillHire follows a three-stage deployment model: prototype (single VM), pilot (two-tier), and production (three-tier / serverless). The table below shows the progression:

Stage	Plan Selected	Components	VM Spec	Reason
Prototype	Single VM	API + NLP + local DB on one VM	4 vCPU, 16 GB RAM, 100 GB SSD	Fastest to deploy; enough for demo and functional testing with <10 concurrent users
Pilot	Two-tier design	App VM + managed DynamoDB + S3 + Lambda NLP	2 vCPU, 4 GB RAM API VM; Lambda 3 GB NLP	Separates NLP compute from API routing; DynamoDB on-demand scales writes; S3 handles file storage
Production	Three-tier + Serverless	CloudFront CDN + ALB + API VM cluster + Lambda NLP + DynamoDB + S3 + QuickSight	2× 2 vCPU API VMs behind ALB; Lambda auto-scales	Elasticity for placement-season burst; zero idle Lambda cost; managed DB and storage remove ops burden

Final VM Plan (Prototype)

AWS EC2 t3.xlarge: 4 vCPU, 16 GB RAM, 100 GB EBS gp3 SSD. Ubuntu 22.04 LTS. Single VM running FastAPI (Python), spaCy NER model, and a local PostgreSQL instance for prototype testing. Reason: the spaCy model requires 6 GB RAM in-process; the 25% buffer pushes total to 16 GB. The 4 vCPU handles NLP computation without queue buildup at 2 RPS prototype load. For pilot and production, NLP migrates to Lambda (3 GB, Docker ECR image) and the API VM downsizes to t3.medium (2 vCPU, 4 GB RAM) behind an Application Load Balancer.

Activity 8: Hypervisor and Virtualization Decision

Context	Selected Approach	Justification
Local prototype/testing	Type 2 hypervisor (VirtualBox / Docker Desktop)	VirtualBox on developer laptop runs Ubuntu 22.04 VM for local API + spaCy testing. Docker Desktop used to build and test the NLP container image before pushing to ECR. Type 2 is sufficient for dev environment — no physical server ownership needed.
Cloud pilot deployment	Cloud-managed virtualization (AWS EC2 / Lambda)	AWS EC2 runs on Type 1 Nitro Hypervisor (AWS KVM-based). The team never manages the hypervisor directly — AWS handles physical isolation, memory protection, and VM lifecycle. Lambda containers use AWS Firecracker microVM (lightweight Type 1) for serverless NLP invocations.
Production growth scenario	Cloud-managed virtualization + Container runtime (AWS ECS / Lambda)	Production NLP runs in Docker containers managed by AWS Lambda (Firecracker). API tier on EC2 Auto Scaling group (AWS Nitro hypervisor). No student-managed hypervisor in production — cloud provider abstracts it entirely.
Module suitable for container	Container runtime (Docker / Lambda Docker Image)	The spaCy NLP Lambda function is the primary container candidate: stateless, short-lived, CPU-intensive per invocation, and requires a specific Python 3.11 + spaCy environment. Containerizing (ECR Docker image) ensures reproducibility and eliminates dependency conflicts across deployments.

Technical Note — Firecracker microVM

AWS Lambda uses Firecracker, a purpose-built microVM technology developed by AWS. Each Lambda invocation runs in its own Firecracker microVM (Type 1 hypervisor subset) for strong isolation, sub-100ms startup, and minimal overhead. This is transparent to the student team — Lambda abstracts microVM management completely.

Activity 9: Snapshot and Clone Strategy

Event	Snapshot or Clone?	Frequency / Timing	Retention / Reason
Before major deployment	Snapshot	Before every production push; triggered manually via AWS CLI / Console	Retain last 3 snapshots (30-day retention). Enables fast rollback if Lambda or API deployment breaks production traffic.
Before DB schema change	Snapshot (EBS) + DynamoDB export	Before any DynamoDB table restructure or GSI addition; S3 DynamoDB Point-in-Time Recovery enabled continuously	DynamoDB PITR retains 35-day restore window. EBS snapshot of prototype VM retained 7 days as fallback for local DB.
Before OS/package update	Snapshot	Before any apt upgrade, Python library version bump, or spaCy model update on the prototype VM	Retain 7 days. If updated spaCy model degrades NER accuracy, snapshot allows instant rollback to previous model version.
Before review/demo	Snapshot	24 hours before each faculty review or placement team demo	Retain until 48 hours after the demo. Ensures the demo environment can be restored identically if a last-minute change breaks functionality.
Create testing environment	Clone	When setting up a staging environment parallel to production	Clone the prototype VM to create a staging VM. Staging tests new spaCy model versions, API endpoint changes, and schema migrations before promoting to production.
Create team demo copy	Clone	Once at start of each sprint for team member individual testing	Each developer gets a clone to run locally in VirtualBox. Prevents dev work from affecting shared staging. Clones are ephemeral — deleted after sprint.

Snapshot vs Clone for SkillHire

Snapshots are used as safety nets before any change that could break the running system. Clones are used to provision parallel environments (staging, dev) without impacting the live prototype. In cloud deployment, EBS snapshots replace manual VM snapshots, and DynamoDB PITR replaces database cloning for recovery.

Activity 10: VM vs Container Decision

Capstone Module	VM / Container / Managed Service	Choice	Reason
Frontend (React SPA)	Managed Service (AWS Amplify + CloudFront CDN)	Managed	Static assets; no server needed. Amplify CI/CD auto-deploys from GitHub. CloudFront serves from edge nodes — no VM overhead for static content.
Backend/API (FastAPI)	VM (EC2 t3.medium prototype) → Container (ECS Fargate in production)	VM → Container	Prototype uses EC2 VM for simplicity. Production migrates API to Docker container on ECS Fargate for portability, auto-scaling, and CI/CD pipeline integration.
NLP Engine (spaCy)	Container (AWS Lambda Docker Image, 3 GB RAM, ECR)	Container	Stateless function; each invocation is isolated. Docker container packages spaCy + pdfplumber precisely. Lambda scales concurrently without a running VM between requests — zero idle cost.
Database	Managed Service (Amazon DynamoDB on-demand)	Managed	No VM or container for database. DynamoDB handles replication, patching, and capacity. On prototype VM: local PostgreSQL for dev/test only.
File storage	Managed Service (Amazon S3 private bucket)	Managed	S3 provides 11-nines durability. Pre-signed URLs allow direct browser-to-S3 upload — NLP Lambda never handles the raw PDF stream, reducing VM memory pressure.
AI/Python module	Container (Lambda Docker image, same as NLP engine)	Container	spaCy inference is packaged in the same Lambda container image as the PDF parser. Separation into a dedicated Lambda function allows independent version updates and isolated error monitoring.
Analytics/dashboard	Managed Service (Amazon QuickSight) + Container (ECS Fargate aggregation job)	Managed + Container	QuickSight provides managed BI without a VM. Periodic aggregation of DynamoDB Streams data runs as a

Capstone Module	VM / Container / Managed Service	Choice	Reason
			scheduled ECS Fargate container task (lightweight, ephemeral).
Notification/background jobs	Managed Service (AWS SNS + Lambda trigger)	Managed + Container	DynamoDB Streams trigger Lambda function (container image) for CloudWatch alerts, SNS email notifications, and scheduled skill-trend recomputation. No dedicated VM required.

Activity 11: Final Infrastructure Recommendation

- 1. VM Sizing.** The SkillHire prototype runs on a single AWS EC2 t3.xlarge instance: 4 vCPU, 16 GB RAM, 100 GB EBS gp3 SSD. This configuration is justified by the CPU score of 10 (driven by the spaCy NLP module at +3 and analytics at +2) and the RAM estimate of 13.75 GB (rounded to 16 GB) dominated by the spaCy model footprint of 6 GB. The 100 GB SSD covers OS (40 GB), local prototype database (10 GB), application logs (5 GB), and a 30% growth buffer (16.5 GB).
- 2. Hypervisor / Virtualization.** Local development uses a Type 2 hypervisor (VirtualBox) hosting Ubuntu 22.04. Cloud deployment uses AWS Nitro Hypervisor (Type 1, managed by AWS) for EC2 instances and AWS Firecracker microVM for Lambda containers. The team never manages the hypervisor directly — cloud-managed virtualization abstracts all hardware isolation.
- 3. Containerization.** The NLP spaCy inference engine is packaged as a Docker container image (Python 3.11, spaCy en_core_web_lg, pdfplumber) and stored in AWS ECR. Lambda runs this container image with 3 GB RAM per invocation. In production, the REST API also migrates to Docker containers on ECS Fargate, enabling CI/CD pipeline deployment from GitHub Actions.
- 4. Storage Plan.** VM disk (100 GB EBS gp3) covers OS and application files only. PDF resumes and gap reports are stored in Amazon S3 (private bucket, SSE-S3, 90-day lifecycle for GDPR). DynamoDB stores structured match results. Logs are retained in CloudWatch (14-day retention). No large files are stored on the VM disk.
- 5. Snapshot / Clone Plan.** EBS snapshots are taken before every deployment, schema change, and OS update (retain 3–7 days). DynamoDB PITR is enabled continuously (35-day restore window). Clones of the prototype VM are created for staging and per-developer test environments at sprint start, and discarded after the sprint.
- 6. VM vs Container Decision.** Frontend: managed (Amplify + CloudFront). Backend API: VM in prototype, container (ECS Fargate) in production. NLP engine: container (Lambda Docker image) from pilot onwards. Database and storage: managed services (DynamoDB, S3). This split assigns the right compute type to each workload — containerized NLP for portability and zero idle cost; managed services for reliability without ops overhead.
- 7. Growth Path.** At 2,000 users and 2 RPS peak, the t3.xlarge prototype is deliberately over-specified to give headroom for load testing. When the product graduates to pilot (institutional deployment), the API VM downsizes to t3.medium (2 vCPU, 4 GB RAM) behind an Application Load Balancer, and Lambda handles NLP auto-scaling elastically from 0 to 500 concurrent invocations.
- 8. Cost Awareness.** Prototype EC2 t3.xlarge: ~\$0.1664/hr on-demand (~\$120/month). Pilot AWS Lambda NLP: 1M free requests/month; beyond free tier: \$0.0000166667 per GB-second × 3 GB × 2 s avg = \$0.0001/invoke. DynamoDB on-demand and S3 (first 5 GB free) keep total pilot cost under \$10/month for ≤500 users.

Carry-Forward to CO2 AT2 and CO2 AT3

Output from CO2 AT1	How It Will Be Used Later
VM sizing values (4 vCPU, 16 GB, 100 GB)	Used as EC2 instance configuration target in CO2 AT2 practical evidence (launch t3.xlarge, attach EBS, verify specs)
Hypervisor/virtualization choice (Nitro + Firecracker)	Used to justify deployment environment selection in CO2 AT2; explains why cloud-managed virtualization was chosen over Type 1/2 self-managed

Output from CO2 AT1	How It Will Be Used Later
Snapshot/clone plan (EBS + DynamoDB PITR)	Used as backup and testing evidence in CO2 AT2; EBS snapshot screenshots and DynamoDB PITR enable/disable proof submitted
VM/container comparison table (Activity 10)	Used in CO2 AT3 infrastructure design case; justifies container choice for NLP module and managed service choice for DB/storage
Final recommendation (8 technical points)	Used as the capstone infrastructure baseline in CO2 AT3; architecture diagram in AT3 must match the sizing and service choices made here

GitHub Submission Checklist

- Completed workbook file (this document — CSA15_CO2_AT1_SkillHire.docx)
- README with capstone title (SkillHire), sizing summary (4 vCPU, 16 GB RAM, 100 GB SSD, Lambda NLP 3 GB), and final recommendation
- Optional architecture diagram showing three-tier + serverless deployment exported as PNG/PDF
- Repository link (CSA15-SkillHire-RegisterNumber/CO2_AT1/) submitted in Google Classroom